

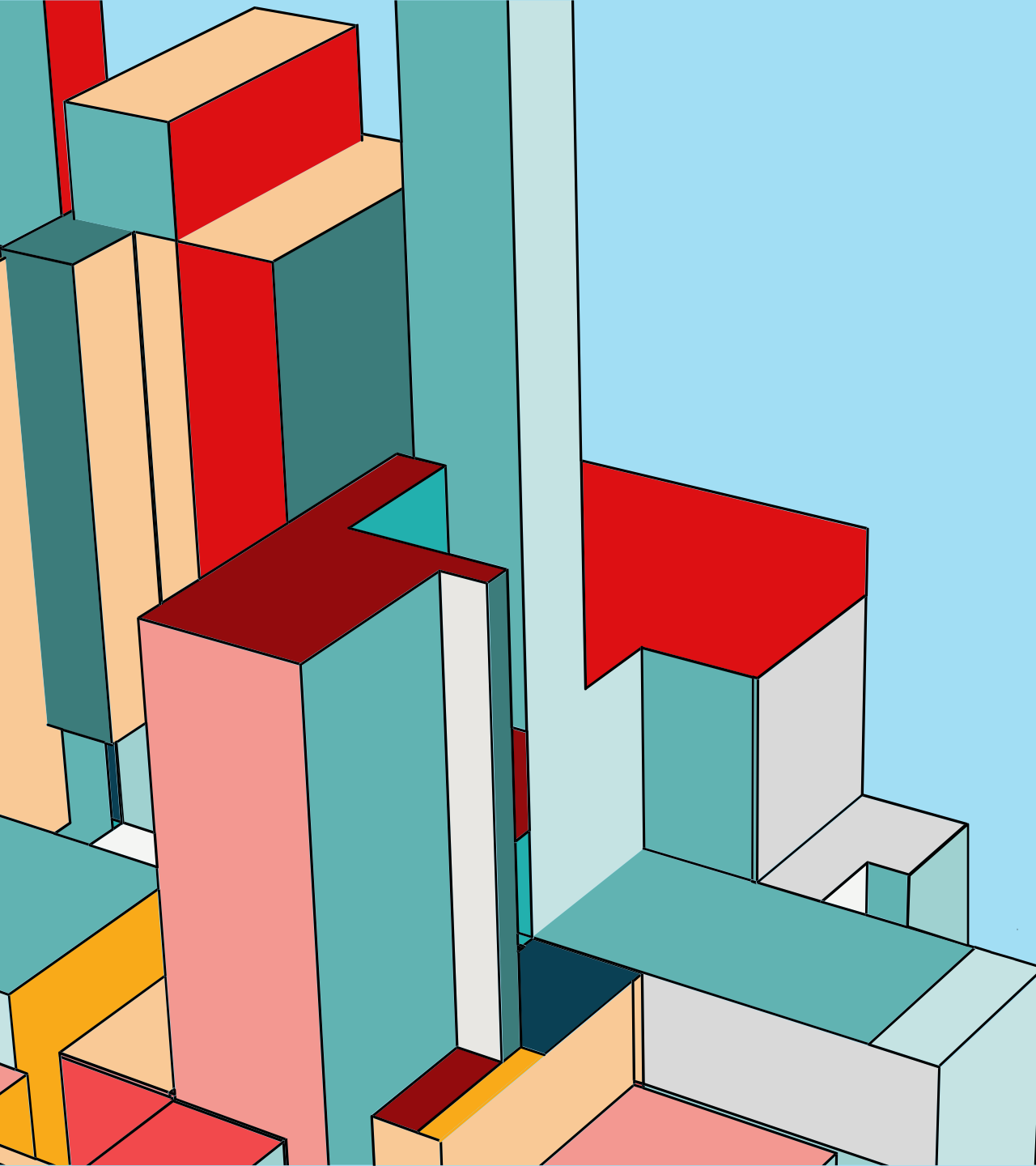
BECOME A
LACROSSE
OFFICIAL



**SIGN
UP TODAY**

**BOYS
LACROSSE**
SECTION 2



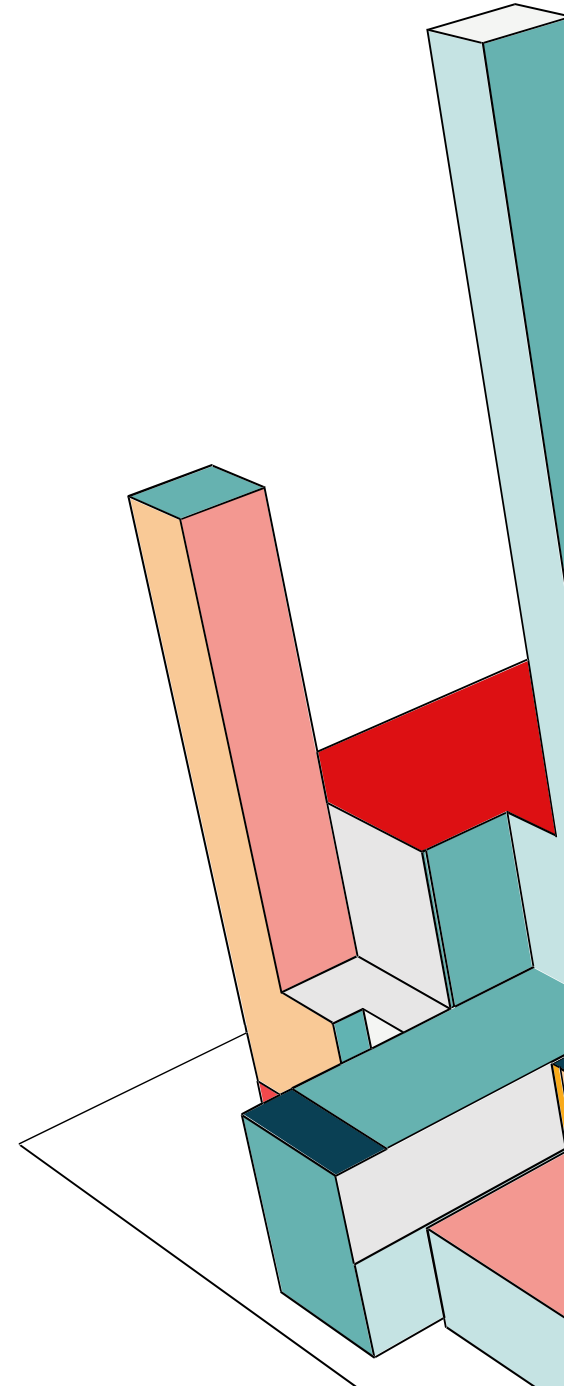


LECTURE 3
**EVERYTHING
EVERYWHERE
ALL AT ONCE:
EXHAUSTIVE
APPROACHES**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

TODAY'S AGENDA

- Recap from last week + HW discussion
- Common problem archetypes
- The “brute force” paradigm
- Applications in searching and sorting
- Exhaustive solutions to other classic problems



DESCRIPTION OF ALGORITHMS

An algorithm is an unambiguous statement of the steps of a process (usually the solution to a problem). It must include a clear declaration of what inputs are valid.

We usually describe algorithms in one of two ways: plain English and pseudocode.

BOTH of these are intended to abstract away some of the implementation details, but they are not the same thing – English descriptions should hide more details, so that we can focus on the critical aspects of the algorithm.

Use a variable x that represents the current value being multiplied.

Set that variable to zero as the first value.

Since the square root of n is a value squared, we check x times x to see if the product is less than or equal to the n value.

Make the increment $+ 1$, giving a $(x+1)$ times $(x+1)$.

Check if the variable x value $+ 1$ multiplied by itself is less than or equal to the n value.

while the product is less than or equal to n , increment the value again and repeat the process.

Check if the next product is less than or equal to n .

If the product is no longer less than or equal to n , take the value of $(x + 1)$ and subtract 1.

This gives us the current x value.

This is the final output.

This “English description” is a direct line-by-line transliteration of code. It reads like someone took a foreign language and ran each individual word through Google Translate... and it's about as useful. Can you figure out what it's trying to do just by reading it?

COMPARE WITH THIS...

Starting at 1, square each positive integer in turn and check if the result is more than n . If it is, stop and return the current integer minus one.

See the difference?

If you are asked for an English description and you find yourself referring to variables, loop increments, or anything similar, the odds are you are worrying too much about implementation details – zoom out and give the big picture description. Finding the right level of abstraction is a critical skill for every computer scientist.

EVEN IN PSEUDOCODE...

The response at right isn't even pseudocode – it's just code!

Just by looking – how sure are you that there are no bugs?

It would be a much better response if we just called Binary Search as if it were a known function (which it is!) instead of writing it out. Alternatively, we can describe its behavior inside the **while** loop – that's the beauty of

pseudocode!

```
def floorRoot(n):  
    if n < 2:  
        return n  
  
    low = 1  
    high = n / 2  
    val = 1  
  
    while low <= high:  
        mid = (low + high) / 2  
        squared = mid * mid  
  
        if squared == n:  
            return mid  
        else if squared < n:  
            val = mid  
            low = mid + 1  
        else:  
            high = mid - 1  
  
    return val
```

ASYMPTOTIC NOTATION

Five types, though the main ones we'll use are big- O and big- Θ . (Maybe a little bit of little- o .)

Most computer scientists will say "big- O " even when they mean big- Θ . (Your professor is definitely guilty of this.)

For non-recursive methods, we find the critical operation (most expensive operation in innermost loop) and count how many times it executes using summations.

For recursive methods, we set up a recurrence relation and then use substitution to solve. (Later on, we'll see another approach to solving recurrences.)

HW QUESTIONS?

Anyone have questions about Homework 1 or Lab 1?
Solutions will be posted next week (when grades are released).

SUBMITTY POLL #1

Is everyone OK with the digital copy of the lab sheet? Is there anyone who would rather have paper copies?



COMMON PROBLEM CATEGORIES IN CS

Search: Given a data structure, determine if a particular value resides within that data structure.

Sorting: Given a collection of comparable objects, put them in order.

String processing: This covers a *lot* of ground. Two of the most common problems here are *substring matching* and *edit distance*.

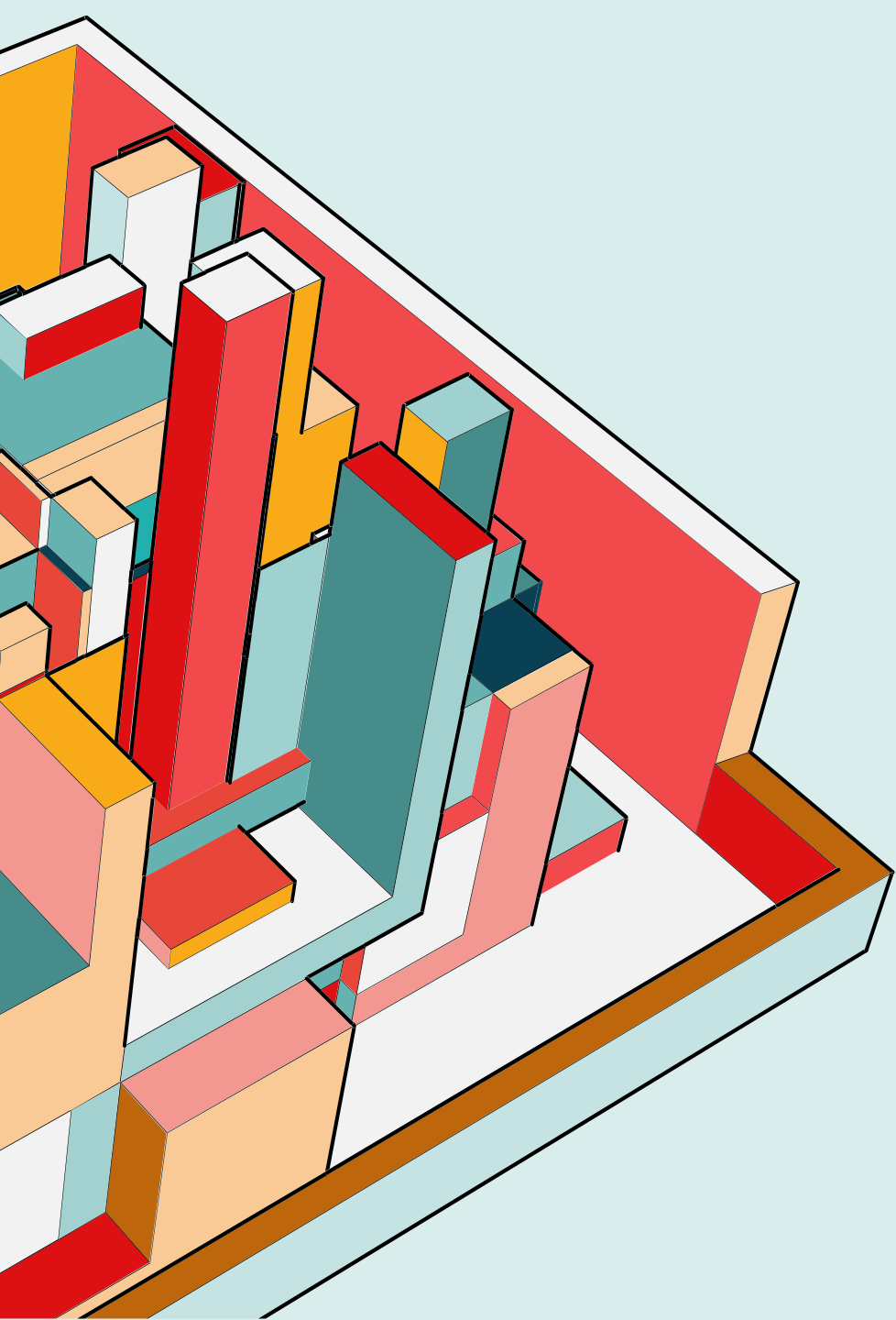
Geometric problems: Problems involving points, lines, planes, polygons, etc. Examples include *closest pair* and *convex hull*.

COMMON PROBLEM CATEGORIES IN CS

Numerical problems: In CS, anything labeled as “numerical” means “continuous”. These often involve optimizing some continuous function under some constraints. Many of our best “solutions” are in fact approximations.

Graph problems: A *huge* category filled with some of the most important problems in the discipline. Prototypical examples include *traveling salesman* and *vertex coloring*.

Combinatorial problems: These are problems involving discrete objects where the potential solution space involves all subsets or sequences of the input, e.g. *subset sum*.



EXHUASTIVE ("BRUTE FORCE") ALGORITHMS

WHAT DO WE MEAN BY “BRUTE FORCE”?

In general, a brute force approach amounts to “guess and check”. It allows a bit of space for us to be clever in making our guesses, but at its core, it means “try everything”.

Sounds REALLY slow. Why would we want to do that?

WELL, IT WORKS...

... on just about everything. It is harder to list the problems for which an exhaustive approach *doesn't* eventually produce the correct answer. (Usually those are the ones where it's hard to know what's correct in the first place.)

And sometimes it's the only thing we know to be correct!

Also, if our problem instances are small enough, it might not be worth the effort to design something more efficient.

Additionally, sometimes we use a multi-stage process, where we use a different approach to break large problems down into smaller ones and then brute force the small ones.

OTHER REASONS

- The exhaustive approach is often the simplest one to state and execute – simplicity has its advantages.
- It can often give us a useful baseline to compare against other algorithms: If your fancy approach can't even beat brute force, why bother with it?
- Finally, it can give you a place to start. In fact, when you have an algorithm design problem that is non-obvious in any way, my strong suggestion is that you begin by jotting down a brute force algorithm – it helps you build up some cognitive momentum.

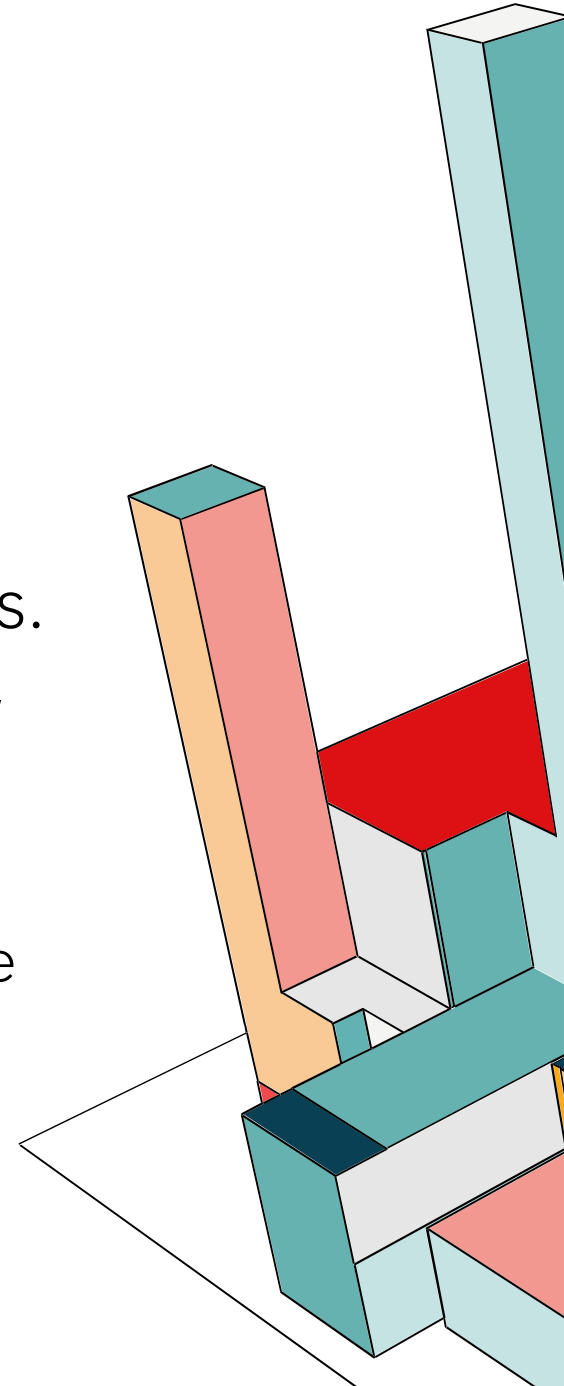
EXAMPLES

We've already seen several examples of brute force algorithms in this class.

Both of the "CS1" algorithms mentioned previously (GCD and n^k) would qualify as exhaustive approaches.

For that matter, the first square root algorithm we saw earlier today (slides 5 & 6) is also a "try everything" approach.

For contrast, the binary search-driven algorithm is not brute force – it's significantly better than trying everything.



A SIMPLE PROBLEM: SEARCH

You have a bunch of boxes of books. You are looking for a particular book. How do you determine if the book is present in one of the boxes or not?

With no other information, you can't do any better than opening the nearest box, checking, and then proceeding to the next box. Brute force is literally the only approach guaranteed to produce the correct answer. Runtime?

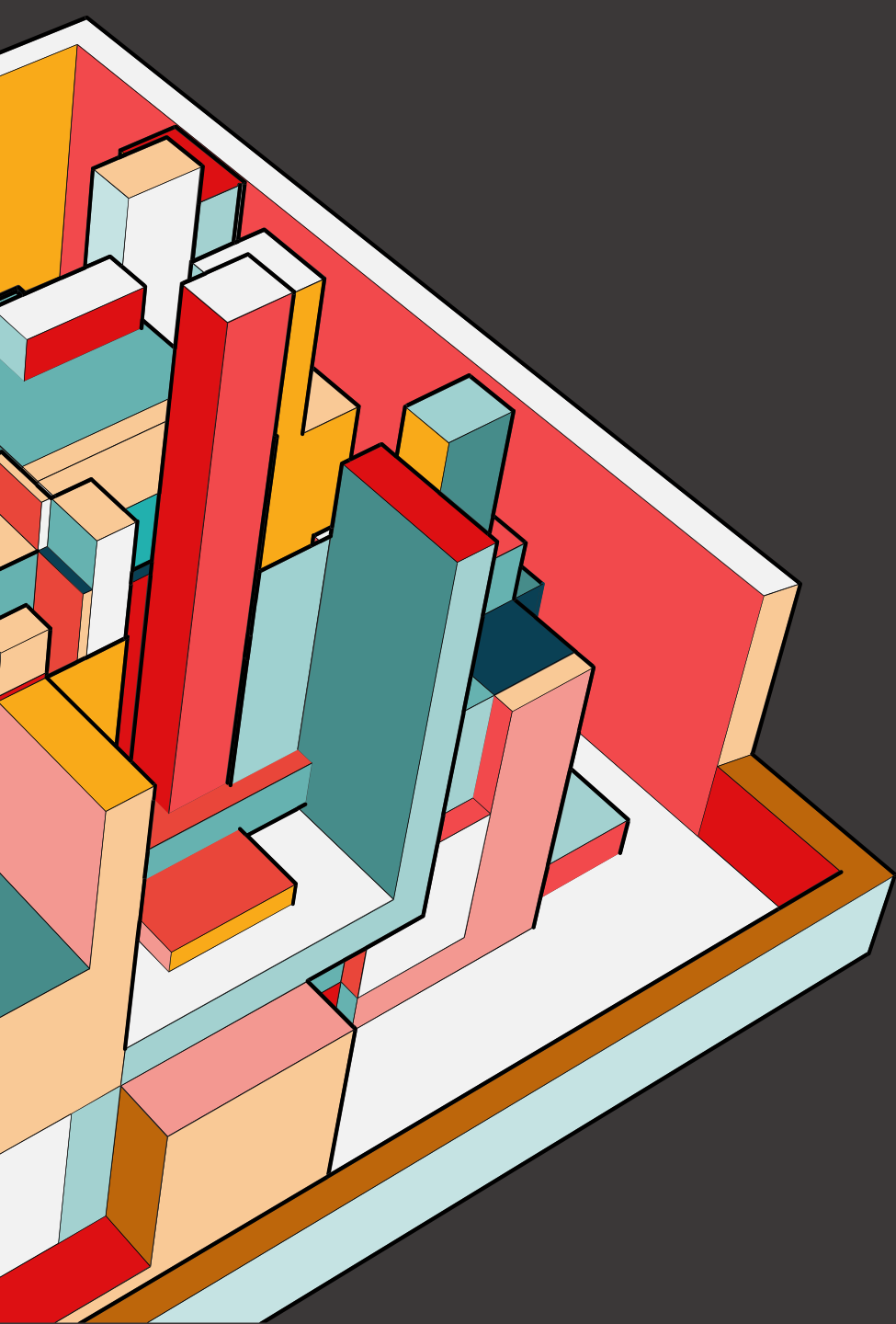
SORT THE BOOKS!

What if you know the books are sorted alphabetically?

We know, of course, that we can use a binary search – and there are even better approaches if we've indexed things correctly.

But we can also improve the humble brute force approach: We still looking in each box in order, but once we pass the alphabetical position of the book we're looking for, we can stop and report that it's not here.

What's the runtime of this improved brute force approach?



QUICK BREAK

SPEAKING OF SORTING



You have a collection of rocks and a basic two-pan balance. You wish to organize the rocks by weight.

What is the most straightforward way to do this?

If you ask folks with no computer science training, you often get one of two ideas:

- Fix things that are out of order as you find them (bubble sort)
- Find the smallest (or largest) item, then the next smallest, etc. (selection sort)

SELECTION SORT

To match the image, we select the largest elements on each pass:

```
def SelectionSort(A,n):  
    for i = (n-1) down to 1:  
        max = i  
        for j = 0 to (i-1):  
            if A[j] > A[i]:  
                max = j  
        swap A[i] and A[max]
```

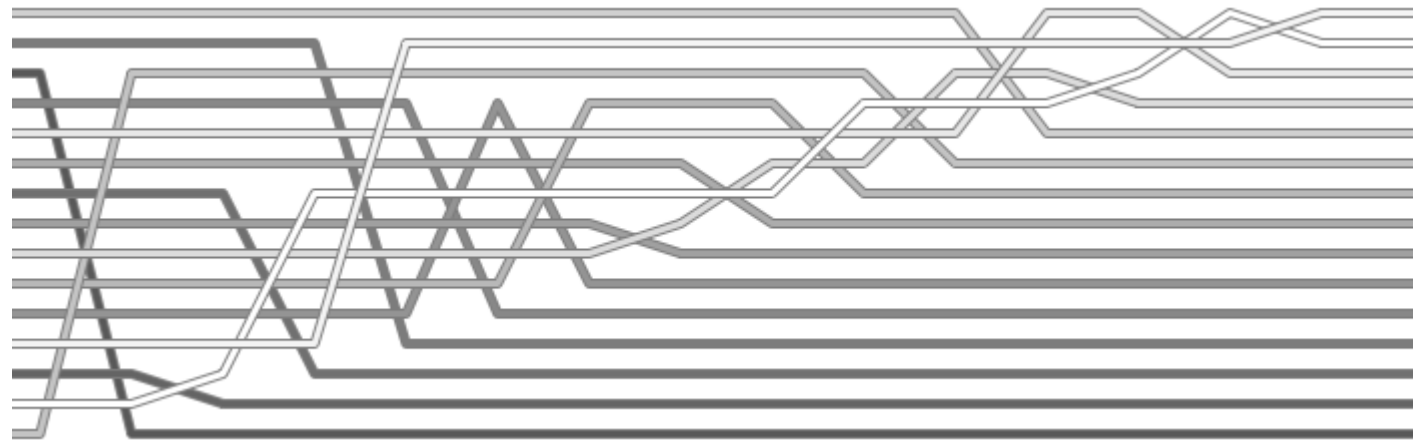


Image credit: Aldo Cortesi, <https://corte.si>

What's our runtime analysis?

- The critical instruction is the comparison.
- Inside loop executes i times; outside loop $n - 1$ times
- So we have the usual $1 + 2 + \dots + (n - 1)$, which is $\Theta(n^2)$.

BUBBLE SORT

```
def BubbleSort(A,n):  
    for i = (n-1) down to 1:  
        for j = 0 to (i-1):  
            if A[j] > A[j+1]:  
                swap A[j] and A[j+1]
```

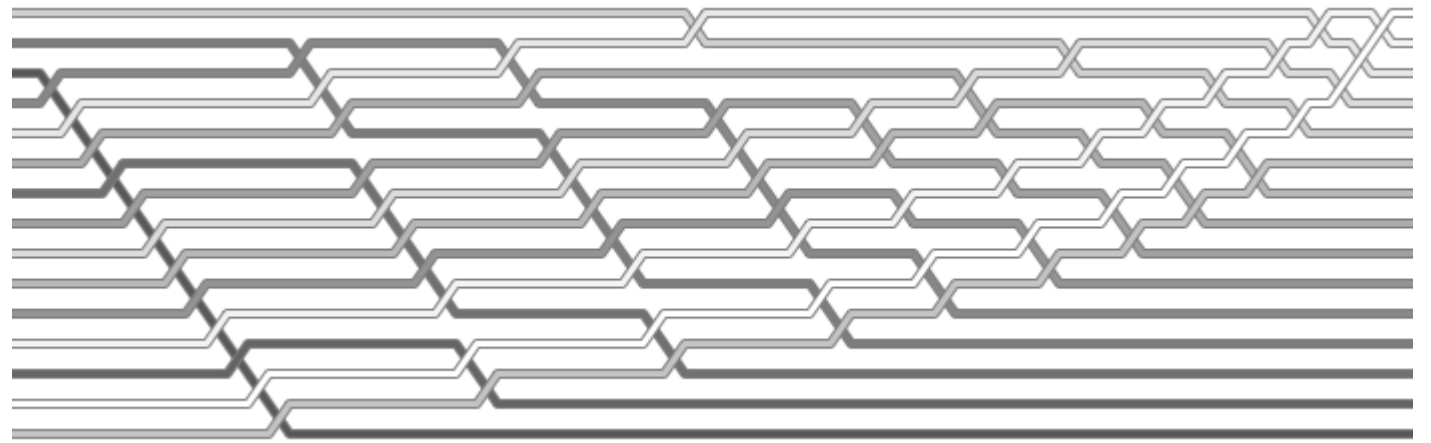


Image credit: Aldo Cortesi, <https://corte.si>

Runtime analysis is essentially the same... BUT – not all quadratic algorithms are created equal!

In fact, bubble sort is one of the worst of the quadratic sorting algorithms because of the number of swaps.

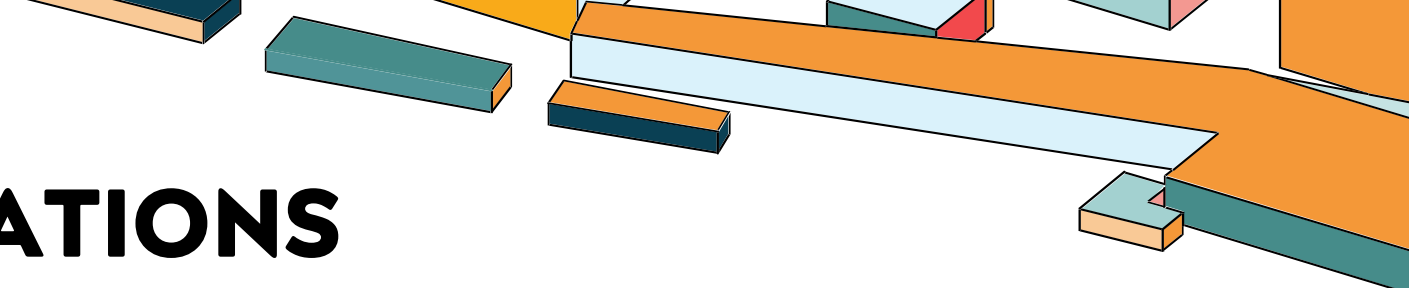
SMALL IMPROVEMENTS

A key thing to remember when working with “naive” algorithms is that we can often make notable improvements on our first attempts with minimal effort.

```
def BubbleSort(A,n):  
    for i = (n-1) down to 1:  
        for j = 0 to (i-1):  
            if A[j] > A[j+1]:  
                swap A[j] and A[j+1]
```

How might we improve this version of bubble sort?

- The most obvious improvement is a check in the outer loop to see if we’ve swapped any elements on this pass. If not, we’re done!
- We can also alternate directions of our “bubble” – this avoids a small item near the end of the list taking *f o r e v e r* to get where it needs to go. The resulting algorithm is called “shaker sort”.



SORTING VISUALIZATIONS

There are a LOT of sorting animations available on the Web, of varying degrees of educational value.

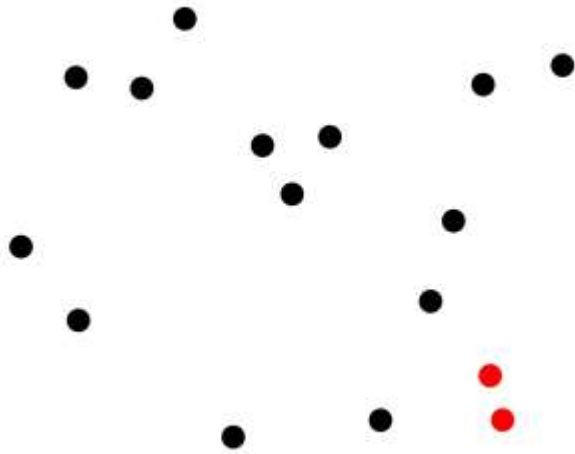
Aldo Cortesi's sortvis.org used to be one of the best, but it seems to be half-defunct.

Right now, I'm partial to David Galles' site at USF (San Francisco, not South Florida):

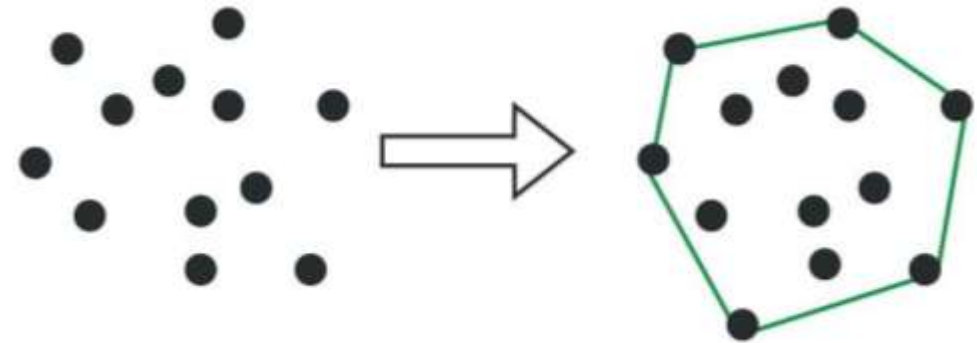
<https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

TWO CLASSIC GEOMETRY PROBLEMS

Closest Pair: Given a collection of points, find the pair of them that are closest together.



Convex Hull: Given a collection of points, find the tightest *convex* boundary that contains all of them.



DO NOW

Write down an algorithm for solving the Closest Pair problem. (If you finish quickly, start thinking about Convex Hull.)

Closest Pair:

- Compute distances of each pair of points, remembering the smallest one we've seen so far.
- When we've done all of them, return the one we are currently remembering.

What's the runtime?

Quadratic number of distances. Distance calculations are linear in the number of dimensions involved. So $\Theta(n^2 d)$.

CONVEX HULL

It may be MUCH less obvious how to even start this one. We'll begin by limiting the scope of the input – let's only deal with points in the Cartesian plane (i.e. 2-D space).

Even still, we need a bit of help from a geometric theorem, one that is obvious in retrospect:

Theorem: The convex hull of a set S of non-collinear 2-D points is a polygon whose vertices are members of S . (If the points are collinear, the polygon collapses to a single line segment.)

A common metaphor is to picture a rubber band stretched around the points – the rubber band will form straight sides between any “outer” points of S .

CONVEX HULL, CONT'D.

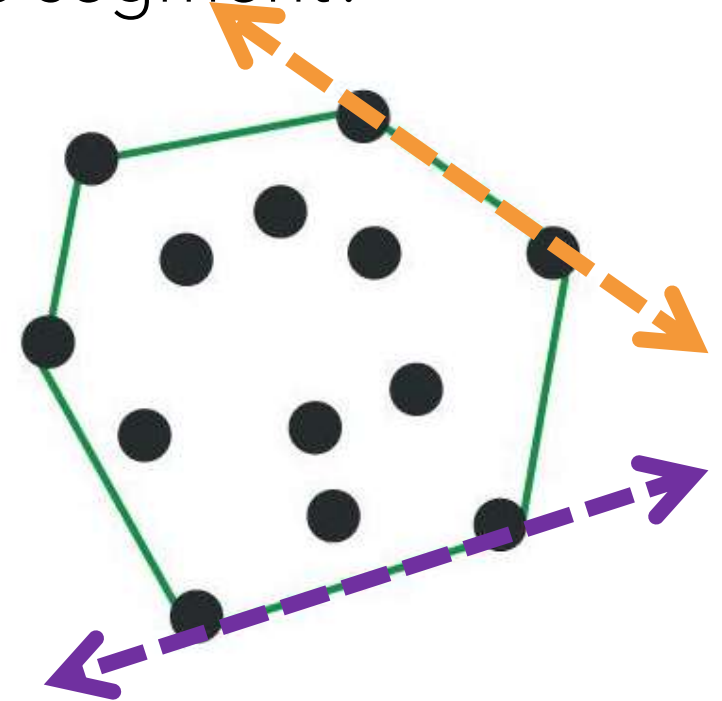
Since the vertices of the polygon are guaranteed to be in S , the sides of the polygon must be line segments joining points of S .

How will the rest of the points relate to this line segment?

Regardless of which line segment we pick...

... all of the points of S land on one side!

Given all of that, can you come up with an algorithm for defining the boundary of the convex hull? Try it now.



SUBMITTY POLL #2

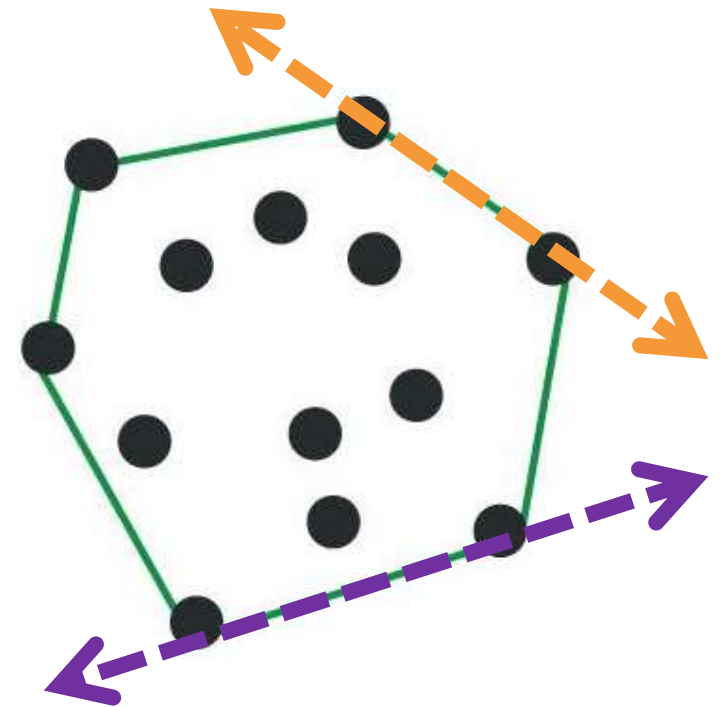
What is the runtime of your Convex Hull algorithm?



CONVEX HULL, CONT'D.

The approach goes like this:

- Iterate over each pair of points.
- Find the equation of the line that connects them ($ax + by = c$).
- Check every OTHER point to see if $ax + by$ is less than or greater than c for all of them.
- If so, this line segment is part of the boundary.



BEFORE YOU LEAVE

